Final Project Report: LLM-Guided Vision-Based Long-Sequence Block Manipulation

Team Number: Team 8
Team Members:
Wang Yisheng – 225040229
Wang Chiyu – 224040323
Yao Siyuan – 121010052
Gu Yingjian - 225040530

Project Repository: https://github.com/chiyuwa/AIR_5051_2026-Team8-FinalProject
Demo Video:
https://www.youtube.com/watch?v=qgsAcTiaF3s&list=PLSZpQ66FAnhWzPikJKi54AtwhPWcKH7qg&index=3

1. Project Overview

This project develops an intelligent robotic manipulation system that can understand natural language instructions, locate a target block through visual perception, and execute a long-sequence pick-and-place task using a robotic arm. The final goal is to allow a user to issue a high-level command such as “detect the block, pick it up, and place it at `place_ready`”, after which the system automatically decomposes the instruction, detects the block, computes its robot-frame position, plans the arm motion, grasps the object, transports it, and releases it at the target location.

The core task performed by our robot is **vision-grounded block manipulation**. Unlike a simple pre-programmed pick-and-place pipeline, the robot does not rely on a manually specified object coordinate. Instead, it first uses a camera to observe the workspace and a YOLO-based visual model, specifically `yolov8s-worldv2.pt`, to identify the target block. The detected bounding box center is then converted from image pixel coordinates into the robotic arm base coordinate system using calibrated linear regression parameters. This allows the robot to infer the physical location of the block and execute a complete grasping sequence.

The main innovation of the project is the integration of a **Large Language Model, visual object detection, and robotic motion execution** into one closed-loop system. A large language model is used as the high-level task planner. It receives a natural language instruction from the user and converts it into a constrained JSON-style task plan. This plan is not directly sent to the robot motors. Instead, it is passed to a local execution layer, which performs normalization, safety checking, and MoveIt-based motion execution. This design makes the robot “smart” in two ways. First, it can understand flexible natural language commands instead of only fixed programmatic inputs. Second, it can ground the command in the real world through the YOLO visual detector and use perception results to guide physical manipulation.

Our system is implemented on top of the existing Sagittarius robotic arm software stack. The lower-level arm driver and trajectory execution are handled by `sdk_sagittarius_arm`, while MoveIt provides motion planning for the `sagittarius_arm` and `sagittarius_gripper` planning groups. On top of this existing stack, we developed a new `sagittarius_llm_control` package that provides the upper-level intelligence layer: natural language input, LLM planning, structured task representation, safety validation, visual detection, and execution orchestration.

The project specifically focuses on long-sequence execution. Instead of only executing a single movement command, the robot performs a sequence of coordinated actions: moving to an observation pose, detecting the target, opening the gripper, moving to a pre-grasp pose, descending to the grasp height, closing the gripper, lifting the object, moving to the placement pose, and opening the gripper to release the block. This long-horizon execution ability is important because many real robotic tasks require multiple dependent steps rather than one isolated motion.

2. Design and Implementation

2.1 Overall System Architecture

The system follows a modular architecture. The large model is responsible for task-level reasoning, while the local ROS-based execution system remains responsible for safety and physical control. The overall pipeline is:

User natural language instruction → LLM task server → DeepSeek planner → structured JSON plan → executor safety checks → vision service if needed → MoveIt motion planning → robotic arm and gripper execution.

This separation is important because the LLM is powerful for semantic understanding and task decomposition, but it should not directly control robot motors. Directly allowing a language model to output low-level motor commands would be unsafe and difficult to verify. Therefore, our system constrains the model output to a predefined task DSL. The local executor then checks whether each step is valid, whether the number of steps is acceptable, whether the target position is within the workspace, and whether the motion step is too large. Only after passing these checks does the system call MoveIt to perform the physical motion.

The main ROS package developed in this project is `sagittarius_llm_control`. It contains the task server, command-line client, planner, executor, vision detector, vision client, service definitions, configuration files, and launch file. The package exposes a natural language service `/sgr532/execute_task`, which receives the user instruction and returns the generated task plan and execution result. It also exposes a visual detection service `/sgr532/detect_block`, which detects the target block and returns its estimated position in the robotic arm base frame.

The corresponding flowchart can be seen in the Appendix.

2.2 Task Planning: The “Brain”

The task planning module is implemented through a DeepSeek-based LLM planner. The user can send a long natural language instruction through the command-line client or the ROS service. The planner then converts this high-level instruction into a structured JSON plan. For example, when the user says “抓取方块并放到 place_ready”, the expected plan contains a `pick_detected_block` step with the target label set to `block` and the placement target set to `place_ready`.

This design allows users to control the robot with natural commands, while keeping the execution format predictable. The planner does not output arbitrary Python code or raw motor commands. Instead, it can only select from predefined actions such as moving to named waypoints, changing orientation presets, opening or closing the gripper, waiting, or invoking the visual grasping skill. This constrained planning strategy improves reliability and prevents unsafe behavior caused by unrestricted LLM output.

A useful feature of the system is the `--plan-only` mode. Before running the real robot, the user can ask the system to only generate the plan without execution. This allows the team to inspect the JSON output and verify whether the LLM correctly understood the instruction. Only after the plan looks reasonable do we remove the `--plan-only` flag and execute the same command on the physical robot. This mode was very useful during debugging because it separated language understanding errors from robot execution errors.

2.3 Visual Perception Module

The perception module is responsible for detecting the target block in the camera image and estimating its position in the robot coordinate frame. We use a YOLO-based visual model, `yolov8s-worldv2.pt`, to perform object detection. The camera publishes images through a ROS image topic, and the YOLO detection node subscribes to the image stream. When the executor needs to grasp a block, it calls the `/sgr532/detect_block` service.

The detector returns the detected object label, confidence score, bounding box center pixel coordinates, and the estimated 3D position used by the robot. The current implementation uses a planar calibration assumption. Specifically, the center pixel coordinates of the bounding box are converted into the robotic arm base frame using linear regression parameters. The conversion follows the idea:

```
base_x = k1 * center_v + b1
base_y = k2 * center_u + b2
base_z = planar_z
```

This method is lightweight and practical for a tabletop block manipulation setting. Although it is not a full 3D reconstruction method, it is sufficient for detecting blocks placed on a relatively fixed planar surface. To improve robustness, the system also provides configurable offsets through `vision.position_offset_xyz`. If the grasp point is consistently shifted in one direction, we can compensate for the bias by slightly adjusting the offset values.

The vision module also supports label normalization. If the model detects labels such as `cube`, `box`, or `carton`, these labels can be mapped to the target category `block` through

the configuration file. This makes the system more flexible when using open-vocabulary or general-purpose detection models.

2.4 Robot Skills: The “Actions”

The robot skills are implemented in the local executor. The executor is responsible for converting the structured task plan into real robot actions. The basic skills include moving the arm to named waypoints, moving to explicit Cartesian positions, switching orientation presets, opening and closing the gripper, waiting, and executing the vision-based grasping skill.

The most important high-level skill in our project is `pick_detected_block`. When this action is executed, the robot first moves to the configured observation waypoint so that the camera can view the workspace. Then it calls the visual detection service to locate the block. After receiving the object position, the robot opens its gripper, moves to a pre-grasp height above the object, descends to the pick height, closes the gripper, lifts the block, moves to the configured placement waypoint, and finally opens the gripper to release the object.

This skill abstracts a complex sequence of physical motions into one reusable action. From the LLM planner’s perspective, it only needs to output `pick_detected_block`. The executor then handles all low-level details, including perception query, coordinate conversion, motion planning, and gripper control. This design makes the system easier to extend. In the future, additional skills such as `pick_colored_block`, `sort_blocks`, or `stack_blocks` can be added using the same architecture.

2.5 Safety and Execution Control

Safety is a central part of the design. The LLM is only used as a high-level planner, while the local executor remains the final authority for robot control. The executor checks the maximum number of steps, workspace boundaries, and maximum translation or rotation per step. If a movement is too large, the executor can split it into smaller MoveIt goals instead of executing a single unsafe long motion.

The configuration file `runtime.yaml` stores important safety and execution parameters, including workspace limits, named waypoints, velocity scaling, acceleration scaling, visual detection parameters, grasp heights, and placement targets. During real hardware testing, we used conservative velocity and acceleration settings to reduce risk. We also followed a step-by-step debugging procedure: first verifying MoveIt and the robot arm, then checking the camera image topic, then testing YOLO detection independently, then validating the LLM-generated plan, and finally running the full physical execution.

This layered safety design improves reliability and makes debugging easier. Even if the LLM produces an unexpected plan, the executor can reject unsafe or invalid commands before they reach the robot.

3. Experiments and Results

3.1 Test Environment

The system was tested in a tabletop manipulation environment using a Sagittarius robotic arm, a gripper, and a camera observing the working area. The target object was a block placed within the calibrated workspace. The robot was required to detect the block, move to it, grasp it, lift it, move to a pre-defined placement pose named `place_ready`, and release the block.

The experiment was designed to evaluate the complete closed-loop capability of the system rather than only isolated modules. Therefore, we tested the following functions: natural language task input, LLM-to-JSON planning, YOLO-based block detection, pixel-to-robot coordinate conversion, MoveIt motion planning, gripper control, and full long-sequence pick-and-place execution.

3.2 Experiment Procedure

We followed a progressive testing strategy. First, we tested the visual detection module independently by calling `/sgr532/detect_block`. This allowed us to verify whether the YOLO model could detect the block and whether the returned position was within the robot's reachable workspace. The expected response included the detected label, confidence score, center pixel coordinates, and estimated position.

Second, we tested the LLM planning module using the `--plan-only` mode. For the instruction "识别方块，抓取后放到 place_ready", the expected generated plan contained the action type `pick_detected_block`, the target label `block`, and the placement target `place_ready`. This confirmed that the LLM planner could correctly transform a high-level natural language command into a structured robotic task.

Third, we tested the complete execution pipeline on the real robot. The system was launched using the integrated launch file, which starts the main components including MoveIt, the camera, the YOLO visual node, and the LLM task server. Then we sent the command "抓取方块并放到 place_ready" through the command-line interface. The robot moved to the observation pose, detected the block, moved to the target position, closed the gripper to grasp the object, lifted it, transported it to the placement area, and released it.

3.3 Results

The final system successfully completed the main project goal: a user can issue a natural language command, and the robot can autonomously perform a vision-guided pick-and-place sequence. The system demonstrates the integration of three key capabilities: language-level task understanding, visual grounding, and physical robot execution.

At the planning level, the LLM was able to decompose long natural language instructions into structured steps. This shows that the system can handle more flexible user input than a traditional hard-coded interface. At the perception level, the YOLO-based detector was able to locate the target block and provide its image-space center coordinates. Through calibration, the system converted these detections into approximate robot-frame positions. At

the action level, the executor successfully coordinated MoveIt arm motion and gripper control to complete the grasping and placement sequence.

The project also demonstrates long-sequence execution. The final behavior is not a single motion command, but a chain of dependent actions. Each step depends on the previous one: the robot must first move to a good observation pose, then detect the block, then compute the grasp position, then approach safely, then grasp, then lift, then move to the placement position, and finally release. The successful execution of this chain shows that the system has achieved a meaningful level of autonomous robotic task execution.

3.4 Challenges and Solutions

One major challenge was connecting the LLM planner with physical robot execution safely. A language model can understand user intent, but its output may be ambiguous or unsafe if directly used for control. To solve this, we introduced a fixed JSON task representation and a local executor. The LLM only outputs a constrained plan, while the executor performs validation and sends final commands to MoveIt. This prevents the model from directly controlling motors and makes the system more robust.

Another challenge was visual grounding. YOLO detects objects in image coordinates, but the robotic arm needs target positions in its own base coordinate frame. To solve this, we used a calibrated linear regression mapping from bounding box center pixels to robot-frame coordinates. Although this method assumes a planar workspace, it is simple and effective for tabletop block manipulation. When a stable offset appeared between the detected grasp position and the actual object location, we adjusted `vision.position_offset_xyz` to compensate for the systematic error.

A third challenge was reliable long-sequence motion execution. Some motions may be too large for a single safe MoveIt step, especially when moving between observation, picking, and placing positions. To address this, the executor includes safety constraints such as maximum translation per step. When a motion exceeds the configured threshold, the executor can automatically split it into smaller segments. This improves execution stability and reduces the risk of sudden or unsafe movement.

A fourth challenge was debugging the full system. Since the final pipeline includes ROS services, camera input, YOLO inference, LLM planning, MoveIt, and gripper control, directly running the full task can make errors difficult to locate. We solved this by designing a clear debugging order. We first tested the camera and visual detection service, then tested LLM planning in `--plan-only` mode, then verified the named waypoints, and finally ran the complete natural language command. This modular debugging process helped us identify problems more efficiently.

3.5 Evaluation Against Grading Criteria

In terms of completeness and feasibility, the project delivers the proposed goal of natural-language-driven, vision-guided block grasping and placement. The system is functional as a complete pipeline from user command to physical execution.

In terms of engineering workload and difficulty, the project integrates multiple complex components: ROS, MoveIt, gripper control, YOLO visual detection, coordinate calibration, LLM planning, JSON task parsing, safety checking, service communication, and launch-file-based system orchestration. The engineering difficulty is significant because the system must work reliably across both software and hardware.

In terms of innovation and cutting-edge technology, the project combines large language models with open-vocabulary-style visual detection and robotic manipulation. Instead of using a simple scripted control pipeline, the system uses the LLM for semantic task planning and YOLO for real-world perception, forming a more intelligent robotic control interface.

In terms of demonstration of intelligence, the robot can interpret a high-level instruction, transform it into executable structured actions, query the visual system when object location is needed, and complete a long-sequence manipulation task. This demonstrates both semantic intelligence and perception-grounded action execution.

In terms of code functionality and quality, the system is organized into clear modules, including the planner, executor, vision detector, vision client, task server, CLI, configuration file, launch file, and ROS service definitions. This modular design improves readability, debugging, and extensibility.

4. Photo and Video Documentation

We show the full task from start to finish in the video.

Final Project Repository:

https://github.com/chiyuwa/AIR5021_RobotManipulation_Project

Final Demo Video Link:

https://www.youtube.com/watch?v=qgsAcTiaF3s&list=PLSZpQ66FAnhWzPikJKi54AtwhPWcKH7qg&index=3

5. Conclusion

This project successfully implements an LLM-guided, vision-based robotic manipulation system for long-sequence block grasping and placement. The robot can receive a high-level natural language command, use a large language model to generate a structured task plan, call a YOLO-based visual detector to locate the block, convert the visual detection result into robot-frame coordinates, and execute the full manipulation sequence using MoveIt and gripper control.

The final achievement is a complete closed-loop system that connects language understanding, visual perception, and physical action. Compared with a traditional scripted robot demo, our system is more flexible because the user can describe the task in natural language. Compared with a pure LLM-based system, our design is safer because the LLM only outputs a constrained plan, while the local executor performs validation and remains

responsible for the final robot commands. Compared with a pure vision-based grasping pipeline, our system is more extensible because the LLM planner can support longer and more diverse task sequences.

There are several directions for future improvement. First, the current system mainly focuses on single-target block grasping. In the future, it can be extended to support multiple objects, object priority, color-based selection, and sequential sorting tasks. Second, the current coordinate mapping is based on planar calibration. A more advanced 3D perception method could improve robustness when the camera angle or object height changes. Third, the system currently does not include automatic failure recovery. Future versions could detect grasp failures and trigger re-detection, re-planning, or retry behaviors. Fourth, a world-state memory module could be added so that the robot can remember previously moved objects and execute more complex long-horizon tasks. Finally, a reusable skill library could be introduced, allowing the LLM to compose higher-level skills such as `pick`, `place`, `sort`, `stack`, and `handover`.

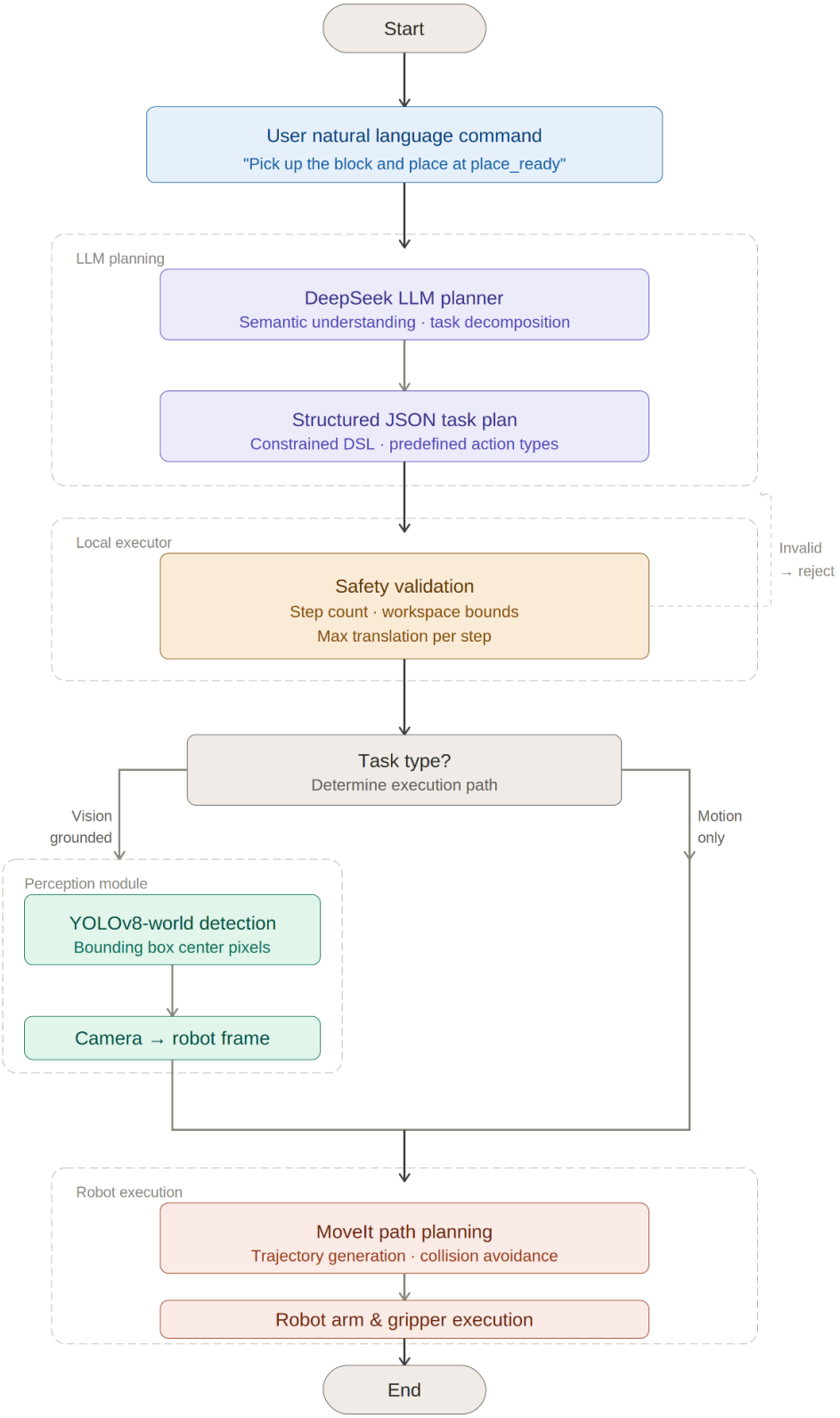
Overall, the project demonstrates a practical and extensible framework for combining LLM task planning, YOLO visual grounding, and ROS/MoveIt-based robotic execution. It shows that large models can be effectively used as high-level robotic planners when their outputs are constrained, validated, and grounded through local perception and control modules.

6. Edge LLM and Hardware Utilization

The large language model was accessed through an API, while robot execution, visual detection, coordinate conversion, and safety checking were handled locally. This hybrid design keeps computationally heavy semantic planning separate from safety-critical physical execution. The robot does not directly follow raw LLM output; instead, the local edge-side system validates and executes only structured, safe commands.

Appendix:

Flowchart



Team Contribution Table

Team Member	Main Contribution
Wang Yisheng	LLM planner, prompt design, code writing, task server
Wang Chiyu	code writing, experiment tuning and debugging, presentation
Yao Siyuan	MoveIt execution, safety checks, document writing, project process coordination
Gu Yingjian	assist in the experimental procedures, demo video, testing